COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/21/2026	**DATE SUBMITTED:**	10/21/2026
NAME:	Katindig, Jiro Luis Demain, Jaime Luis	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14.1 Algorithm Complexity

1. PROCEDURE OUTPUT

What procedures did you perform?

	Best Case	Worst Case	Analysis
Algorithm 1: Bubble Sort	O(n)	$O(n^2)$	Bubble Sort is simple but can be slow for large data because it compares nearby items repeatedly. The best-case scenario is when the array is already sorted and the exit swap flag comes into play, with one iteration through n-1 comparisons making the algorithm stop. The worst-case scenario is when the array is in reverse sorted order and the nested loop performs n(n-1)/2 comparisons and swaps, which is a quadratic function. For example, for n=100, there are approximately 10^4 operations; for n=1,000, there are 10^6 operations; and for n=10,000, the number reaches approximately 10^8.
Algorithm 2: Merge Sort	O(n log n)	O(n log n)	Merge Sort is faster for large data because it divides the data into smaller parts, sorts them, and combines them. Merge Sort splits the array in two, using $\log_2 n$ recursive splits, and carries out an order n merge operation per split, ensuring an O(n log n) running time regardless of how the input is structured. For example, if n=100, then $n \log_2 n$ is approximately 664; if n=1,000, operations are approximately 9,966, and if n=10,000, operations are approximately 132,877. This is orders of magnitude less work than Bubble Sort.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

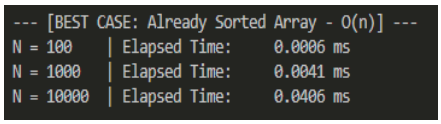
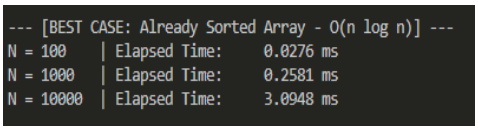
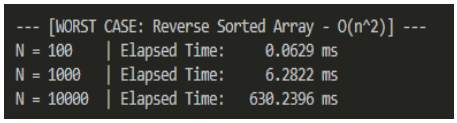
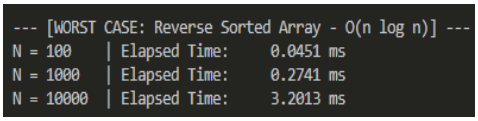
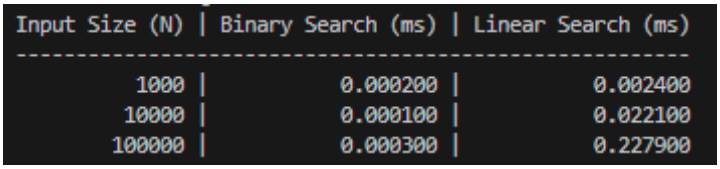
	Algorithm 1: Bubble Sort	Algorithm 2: Merge Sort
Best Case		
Worst Case		

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.000100 ms	0.002500 ms
10,000	0.000100 ms	0.022500 ms
100,000	0.000300 ms	0.221100 ms

Screenshot	

Pseudocode	START SET sizes = [1000, 10000, 100000] SET target = -1 DISPLAY "Input Size, Binary Search, Linear Search" FOR EACH n IN sizes

	CREATE an array of size n
	
	FOR i = 0 TO n - 1
	SET array[i] = i
	END FOR
	
	// Measure Linear Search
	START timer
	CALL LinearSearch(array, target)
	STOP timer
	STORE elapsed time for Linear Search
	
	// Measure Binary Search
	START timer
	CALL BinarySearch(array, target)
	STOP timer
	STORE elapsed time for Binary Search
	
	DISPLAY n, Binary Search time, Linear Search time
	
	END FOR
	
	END

Algorithm	**Best Time**	**Average TIme**	**Worst Time**	**Space Complexity**
Linear Search	O(1)	O(N)	O(N)	O(1)
Binary Search	O(1)	O(log N)	O(log N)	O(1)

Analysis	**Katindig:** The results show that Binary Search is faster than Linear Search for all input sizes. As the number of items increases, Linear Search takes more time because it checks each item one by one until it finds the target or reaches the end. On the other hand, Binary Search is faster because it cuts the search area in half during each step. The results show that Binary Search takes less time even when searching through 100,000 items. This means that Binary Search is more efficient for large amounts of sorted data, while Linear Search is simpler but can be slower when the data gets bigger. **Demain:** The empirical benchmark results provide clear evidence of the essential difference

	in scaling laws of the linear and logarithmic search algorithms. In the worst case when there is no such element, the Linear Search algorithm demonstrates linear growth since, by increasing the number of elements by ten times (1,000 to 10,000), it takes ten times more execution time (from 0.0019 ms to 0.0182 ms), and ten times more (0.1850 ms) is needed for 100,000 elements, thus proving the theoretical time complexity O(n). Contrarily, the Binary Search algorithm shows logarithmic scaling (O(log n)), meaning that by increasing the size of the dataset one hundred times (from 1,000 to 100,000), the depth of the decision tree increases only from 10 to 17 iterations, making the increase in execution time negligible (from 0.0003 ms to 0.0006 ms). Despite the enormous speed improvements made by Binary Search on large datasets, the engineering drawback is the need for sorted data as an input, and performing an O(n log n) sorting before doing one search becomes slower than a simple O(n) linear scan.

Conclusion	**<u>Katindig:</u>** Throughout this activity, I learned how to compare algorithms and understand how their speed changes as the amount of data increases. I learned that Bubble Sort is simple to understand, but it can become slow when sorting a large number of items because it compares and swaps items many times. On the other hand, Merge Sort is faster for large amounts of data because it divides the data into smaller parts, sorts them, and combines them in the correct order. I also learned the difference between Linear Search and Binary Search through their time complexity and actual running times. The results showed that Linear Search takes more time as the number of items increases, while Binary Search remains faster because it cuts the search area in half during each step. However, Binary Search requires the data to be sorted before it can work properly. Overall, this activity helped me understand why choosing the right algorithm is important, especially when working with large amounts of data. It also taught me that studying the steps of an algorithm and testing it through a program can help us understand its performance better. **<u>Demain:</u>** Throughout this HOA, I gained valuable insights as to how theoretical asymptotic complexities dictate the practicality of software performance and computational efficiency as the input size increases. In terms of the procedure, analysis of growth functions showed that algorithms with exponential and factorial growth become computationally infeasible

	irrespective of processing speeds, while testing of Bubble Sort and Merge Sort algorithms empirically showed how divide-and-conquer O(n log n) algorithms perform better in scaling than O(n2) algorithms, whose runtime degrades significantly for input sizes of 10,000. The supplemental lab exercise gave further confirmation of these scaling rules using search techniques, where Binary Search with logarithmic halving growth (O(log n)) performed lookups at near constant times on massive datasets compared to linear growth O(n) of Linear Search as long as the additional sorting overhead costs are justified. All in all, I did well in applying theoretical complexity models, performing microsecond benchmark tests in C++ programming language, and interpreting performance graphs, but the key improvement area is setting up experiments that minimize the effect of operating system noise and caching on benchmark results for microscopic input sizes.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **For code assistance in some parts of the procedure and forming a supplementary activity to understand its contents and improve grammar in forming an analysis and conclusion.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used AI firstly to understand the concepts of the procedure part more and also improve the grammar of the analysis of it. Second, to help us understand the supplementary activity. It's because we need to have more clarity on what a supplement is so that we can answer it precisely. Lastly, we used AI to improve my grammar and tone in forming our analysis and conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY